

ΕΡΓΑΣΙΑ ΜΑΘΗΜΑΤΟΣ

(προθεσμία 08/01/2026)

Επιλογή 1

Δείτε τη δημοσίευση που έχω ανεβάσει στο eclass σχετικά με skyline queries.

Στόχος είναι να υλοποιήσετε είτε σε Apache Flink τους 3 Αλγόριθμους που απεικονίζονται στο Figure 3 της δημοσίευσης. Οι 3 αλγόριθμοι μοιάζουν αρκετά και η υλοποίησή σας λογικά θα διαφέρει λίγο μεταξύ τους (αλλιώς κάτι δεν κάνετε σωστά).

Ο αλγόριθμός σας θα πρέπει να λειτουργεί κατανεμημένα, με επίπεδο παραλληλισμού μεταβλητό (παράμετρο του προγράμματος), και όχι απλά ίσο με 1 (μη παράλληλο). **Η σωστή υλοποίηση είναι πολύ σημαντικό κομμάτι του βαθμού.**

Το πρόγραμμα πρέπει να διαβάζει διαρκώς από το Kafka tuples και να κάνει κάποια επεξεργασία (αλλά να μη τυπώνει) το skyline πάνω σε αυτά.

Το πρόγραμμά σας θα πρέπει να διαβάζει από κάποιο άλλο Kafka topic ερωτήματα (τα οποία μπορεί να είναι απλά ένας αριθμός), και μόνο τότε θα γράφετε ως έξοδο το συνολικό skyline μέχρι εκείνη τη στιγμή σε κάποιο νέο output Kafka topic.

Θα χρειαστεί να σκεφτείτε πόση επεξεργασία θα πρέπει να γίνεται όταν διαβάζετε tuples, ώστε να μεγιστοποιηθεί ο ρυθμός εξυπηρέτησης δεδομένων της εφαρμογής σας.

Τα δεδομένα εισόδου θα πρέπει να είναι πολύ μεγάλα (> 10.000.000 tuples) σε μέγεθος, ώστε να μπορέσετε να ελέγξετε την κλιμακοσιμότητα του προγράμματός σας.

Ο μέγιστος βαθμός στην ΕΠΙΛΟΓΗ 1 είναι το 11/10. Το 80% του βαθμού θα είναι το παραπάνω ζητούμενο, αν λειτουργεί σε 2διστατα δεδομένα. Το 20% του βαθμού θα είναι αν το πρόγραμμά σας είναι παραμετροποιήσιμο. Αν τα δεδομένα σας δηλαδή διαβάζονται ως tuples, να μπορείτε να περάσετε το όνομα του Kafka topic ως παράμετρο, πιθανώς (αν το χρειαστείτε) το domain κάθε διάστασης, να μπορείτε να ορίσετε τον αλγόριθμο skyline που θέλετε να τρέξετε ως παράμετρο, και να μπορείτε να τρέχετε τον κώδικά σας για δεδομένα διάστασης και μεγαλύτερης του 2.

Πρέπει να χρησιμοποιήσετε το streaming API του Flink.

Επιλογή 2

Στην εργασία σας θα πρέπει να φτιάξετε ένα πρόγραμμα που να υλοποιεί τον αλγόριθμο του BUC (Bottom-Up Computation of Sparse Iceburg Cubes). Στην εργασία σας μπορείτε να κάνετε κάποιες υποθέσεις (περιγράφονται παρακάτω) που διευκολύνουν την υλοποίηση του αλγόριθμου.

Δεδομένα εισόδου

Το fact table είναι σε ένα ASCII αρχείο με πολλές εγγραφές, η κάθε μία από τις οποίες είναι σε ξεχωριστή γραμμή. Κάθε εγγραφή αποτελείται από D ακραίους (οι διαστάσεις) και 1 measure (ακέραιος). Σε κάθε πλειάδα τα πεδία χωρίζονται από τον κενό χαρακτήρα. Μπορείτε να υποθέσετε ότι οι διακριτές τιμές (cardinality) κάθε διάστασης δεν ξεπερνάνε τις 10000. Το αρχείο αυτό θα πρέπει να το δημιουργείται εσείς από ένα βοηθητικό πρόγραμμα, το οποίο θα παίρνει ως είσοδο

- Τον αριθμό των διαστάσεων
- Το cardinality σε κάθε διάσταση
- Τον αριθμό των πλειάδων του fact table

Πρόγραμμα

Το πρόγραμμά σας θα πρέπει να δημιουργεί τόσα αρχεία όσες οι όψεις του iceberg κύβου. Κάθε αρχείο θα περιέχει τα tuples της αντίστοιχης όψης, όπως αυτά υπολογίζονται από τον αλγόριθμο του BUC. Τα ονόματα των διαστάσεων των δεδομένων θεωρήστε ότι είναι γράμματα (A, B, C, D, κτλ), οπότε το αρχείο για το view ABD θα πρέπει να έχει το αντίστοιχο όνομα (ABD). Το αρχείο για το group by none θα πρέπει να έχει το όνομα none.

Σκοπός σας, πέραν της ορθότητας του προγράμματός σας, είναι να κάνετε όσον το δυνατόν περισσότερες βελτιστοποιήσεις στην υλοποίηση του αλγόριθμου. Μπορεί να έχετε πχ multi-threaded υπολογισμό.

Θα πρέπει να δημιουργήσετε πειράματα αντίστοιχα με τα πειράματα της δημοσίευσης, και να παράξετε τις αντίστοιχες γραφικές παραστάσεις. Αν δείτε τις διαφάνειες, πράγματα που μεταβάλλονται είναι ο αριθμός των διαστάσεων, το cardinality, το minimum support και ο αριθμός των πλειάδων του fact table. Προφανώς, επειδή η δημοσίευση ήταν πριν από 25 χρόνια, θα μπορούσατε να τρέξετε πολύ μεγαλύτερα πειράματα από τη δημοσίευση.

Επιλογή 3

Στην εργασία σας θα πρέπει να φτιάξετε ένα πρόγραμμα που να υλοποιεί τον αλγόριθμο του Dwarf. Στην εργασία σας μπορείτε να κάνετε παρόμοιες υποθέσεις με αυτές που περιγράφονται στην Επιλογή 2.

Βαθμολόγηση για τις Επιλογές 2 και 3

Κώδικας που λειτουργεί μόνο υπολογίζει μόνο πράγματα στη μνήμη και λειτουργεί μόνο αν το αποτέλεσμα και το fact table χωράνε στη μνήμη θα έχει μέγιστο βαθμό το 9/10. Κώδικας που λειτουργεί και αν το fact table ή/και το τελικό Dwarf δε χωράνε στη

μνήμη θα έχει μέγιστο βαθμό 12/10. Η ταχύτητα του αλγόριθμου και οι βελτιστοποιήσεις που θα κάνετε αποτελούν πολύ σημαντικό μέρος της βαθμολογίας. Πολύ καλές και έξυπνες βελτιστοποιήσεις θα λάβουν bonus.

Δείτε το πώς δουλεύουν τα memory mapped αρχεία, πώς μπορείτε να γράφετε τα views σε binary format, σκεφτείτε πώς ΔΕ θα γράφετε κάθε tuple σε δίσκο αλλά θα γράφετε στο δίσκο όταν γεμίζει κάποια σελίδα, κτλ.

Γλώσσα Υλοποίησης θα πρέπει να είναι η C, ή η C++, ή η Java.

Σημαντικές Επισημάνσεις

Η χρήση κώδικα (μικρού ή μεγάλου) από το internet που δεν είναι δικός σας οδηγεί απευθείας σε μηδενισμό της άσκησης.